

Accepted 96 / 96 testcases passed

AnaghaAvinash submitted at Apr 15, 2026 15:20

Analysis

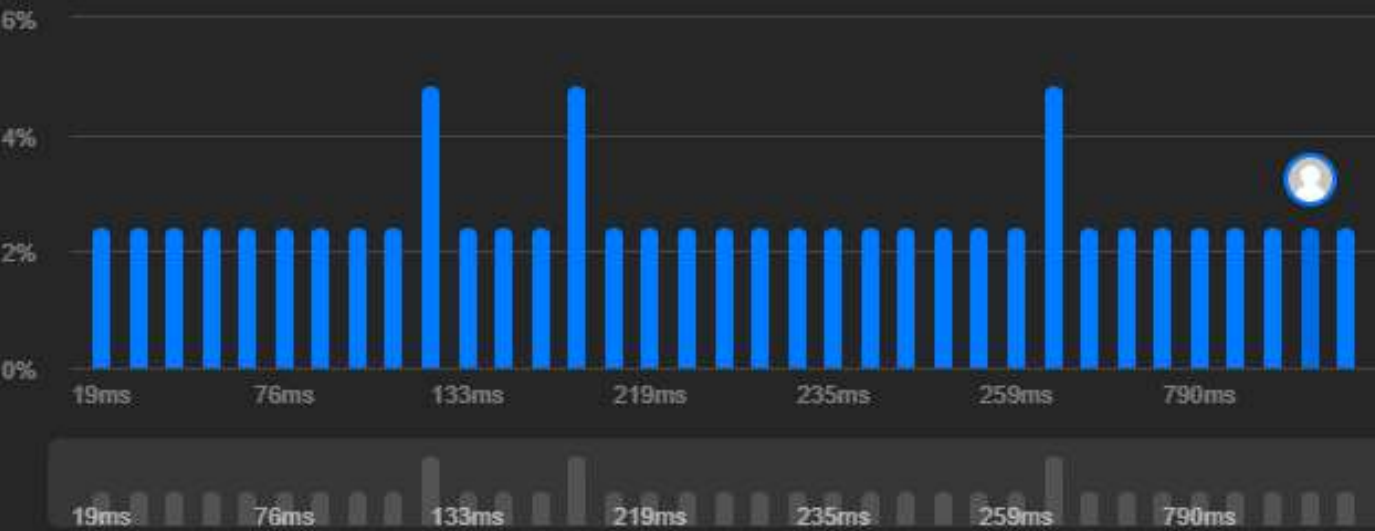
Solution

Runtime

910 ms | Beats 9.76%

Memory

109.32 MB | Beats 26.83%



</> Code

C Auto

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4
5  typedef struct {
6      int to;
7      int time;
8  } Edge;
9
10 typedef struct {
11     int cost;
12     int city;
13     int time;
14 } State;
15
16 typedef struct {
17     State *arr;
18     int size;
19     int capacity;
20 } MinHeap;
21
22 void swap(State *a, State *b) {
23     State temp = *a;
24     *a = *b;
25     *b = temp;
26 }
27
28 MinHeap* createHeap(int capacity) {
29     MinHeap* heap = (MinHeap*)malloc(sizeof(MinHeap));
30     heap->arr = (State*)malloc(sizeof(State) * capacity);
31     heap->size = 0;
32     heap->capacity = capacity;
33     return heap;
34 }
35
```

</> Code

C Auto

```
34 }
35
36 void push(MinHeap* heap, State s) {
37     int i = heap->size++;
38     heap->arr[i] = s;
39     while (i > 0 && heap->arr[(i-1)/2].cost > heap->arr[i].cost) {
40         swap(&heap->arr[i], &heap->arr[(i-1)/2]);
41         i = (i-1)/2;
42     }
43 }
44
45 State pop(MinHeap* heap) {
46     State root = heap->arr[0];
47     heap->arr[0] = heap->arr[--heap->size];
48     int i = 0;
49     while (1) {
50         int left = 2*i+1, right = 2*i+2, smallest = i;
51         if (left < heap->size && heap->arr[left].cost < heap->arr[smallest].cost) smallest = left;
52         if (right < heap->size && heap->arr[right].cost < heap->arr[smallest].cost) smallest = right;
53         if (smallest == i) break;
54         swap(&heap->arr[i], &heap->arr[smallest]);
55         i = smallest;
56     }
57     return root;
58 }
59
60 int minCost(int maxTime, int** edges, int edgesSize, int* edgesColSize, int* passingFees, int passingFeesSize) {
61     // adjacency list stored by value
62     Edge adj[passingFeesSize][edgesSize];
63     int adjCount[passingFeesSize];
64     for (int i = 0; i < passingFeesSize; i++) adjCount[i] = 0;
65
66     for (int i = 0; i < edgesSize; i++) {
67         int u = edges[i][0], v = edges[i][1], t = edges[i][2];
68         adj[u][adjCount[u]++] = (Edge){v, t};
69         adj[v][adjCount[v]++] = (Edge){u, t};
70     }
71 }
```

```

60 int minCost(int maxTime, int** edges, int edgesSize, int* edgesColSize, int* passingFees, int passingFeesSize) {
61     // adjacency list stored by value
62     Edge adj[passingFeesSize][edgesSize];
63     int adjCount[passingFeesSize];
64     for (int i = 0; i < passingFeesSize; i++) adjCount[i] = 0;
65
66     for (int i = 0; i < edgesSize; i++) {
67         int u = edges[i][0], v = edges[i][1], t = edges[i][2];
68         adj[u][adjCount[u]++] = (Edge){v, t};
69         adj[v][adjCount[v]++] = (Edge){u, t};
70     }
71
72     // DP table
73     int** dp = (int**)malloc(sizeof(int*) * passingFeesSize);
74     for (int i = 0; i < passingFeesSize; i++) {
75         dp[i] = (int*)malloc(sizeof(int) * (maxTime+1));
76         for (int j = 0; j <= maxTime; j++) dp[i][j] = INT_MAX;
77     }
78
79     MinHeap* heap = createHeap(edgesSize * passingFeesSize * (maxTime+1));
80     push(heap, (State){passingFees[0], 0, 0});
81     dp[0][0] = passingFees[0];
82
83     while (heap->size > 0) {
84         State cur = pop(heap);
85         int cost = cur.cost, city = cur.city, time = cur.time;
86
87         if (city == passingFeesSize-1) {
88             // free memory before returning
89             for (int i = 0; i < passingFeesSize; i++) free(dp[i]);
90             free(dp);
91             free(heap->arr);
92             free(heap);
93             return cost;
94         }
95     }

```

</> Code

C v Auto

```
83 while (heap->size > 0) {
84     State cur = pop(heap);
85     int cost = cur.cost, city = cur.city, time = cur.time;
86
87     if (city == passingFeesSize-1) {
88         // free memory before returning
89         for (int i = 0; i < passingFeesSize; i++) free(dp[i]);
90         free(dp);
91         free(heap->arr);
92         free(heap);
93         return cost;
94     }
95
96     for (int i = 0; i < adjCount[city]; i++) {
97         Edge e = adj[city][i];
98         int newTime = time + e.time;
99         if (newTime <= maxTime) {
100             int newCost = cost + passingFees[e.to];
101             if (newCost < dp[e.to][newTime]) {
102                 dp[e.to][newTime] = newCost;
103                 push(heap, (State){newCost, e.to, newTime});
104             }
105         }
106     }
107 }
108
109 // free memory before returning
110 for (int i = 0; i < passingFeesSize; i++) free(dp[i]);
111 free(dp);
112 free(heap->arr);
113 free(heap);
114
115 return -1;
116 }
```